```
import pandas as pd

FILE = "/content/drive/MyDrive/Business_Intelligence/BI_Retail_Case_Study.xlsx"   # no trailing slash
xls = pd.ExcelFile(FILE)
print(xls.sheet_names)
```

```
['SalesRaw', 'DimProduct', 'DimCustomer', 'DimStore', 'DimEmployee', 'DimDate', 'Targets']
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

FILE = "/content/drive/MyDrive/Business_Intelligence/BI_Retail_Case_Study.xlsx"

# See all sheets
xls = pd.ExcelFile(FILE)
print(xls.sheet_names)

# Load each sheet (adjust names to what you see)
sales   = pd.read_excel(FILE, sheet_name="SalesRaw")
products= pd.read_excel(FILE, sheet_name="DimProduct")
stores  = pd.read_excel(FILE, sheet_name="DimStore")
targets = pd.read_excel(FILE, sheet_name="Targets")
# Employees / Customers may be in SalesRaw or a Dim table — check.
```

```
['SalesRaw', 'DimProduct', 'DimCustomer', 'DimStore', 'DimEmployee', 'DimDate', 'Targets']
```

Q1: Spearman correlation, the top products, which products are high-sales/low-profit

```
df = sales.merge(products, on="ProductID", how="left")

prod = df.groupby(["ProductID","ProductName","Category"]).agg(
    TotalNetSales = ("NetSales","sum"),
    TotalQty      = ("Quantity","sum"),
).reset_index()

# Cost = UnitCost * Quantity  (per transaction, summed)
df["TotalCost"] = df["UnitCost"] * df["Quantity"]
cost = df.groupby("ProductID")["TotalCost"].sum().reset_index()
prod = prod.merge(cost, on="ProductID")

prod["GrossProfit"] = prod["TotalNetSales"] - prod["TotalCost"]
prod["ProfitMargin"] = prod["GrossProfit"] / prod["TotalNetSales"]

# Rankings
prod["SalesRank"]  = prod["TotalNetSales"].rank(ascending=False)
prod["ProfitRank"] = prod["GrossProfit"].rank(ascending=False)
prod["RankGap"]    = prod["SalesRank"] - prod["ProfitRank"]

# Correlation between the two rankings
print(prod[["TotalNetSales","GrossProfit"]].corr(method="spearman"))

# High sales, low profit
high_sales_low_profit = prod[(prod.SalesRank <= 10) & (prod.ProfitRank > 20)]
# Low sales, high profit
low_sales_high_profit = prod[(prod.SalesRank > 20) & (prod.ProfitRank <= 10)]
```

```
               TotalNetSales  GrossProfit
TotalNetSales        1.00000      0.98833
GrossProfit          0.98833      1.00000
```

Q2: correlation values, the discount-band table

```
sales_with_cost = sales.merge(products[['ProductID', 'StandardUnitPrice']], on='ProductID', how='left')
sales_with_cost['TransactionCost'] = sales_with_cost['Quantity'] * sales_with_cost['StandardUnitPrice']
sales_with_cost['GrossProfit'] = sales_with_cost['NetSales'] - sales_with_cost['TransactionCost']

q2 = sales_with_cost[["DiscountPct","DiscountAmount","GrossSales","NetSales",
            "Quantity","GrossProfit"]].copy()

print(q2.describe())
```

```
print(q2.corr(numeric_only=True))

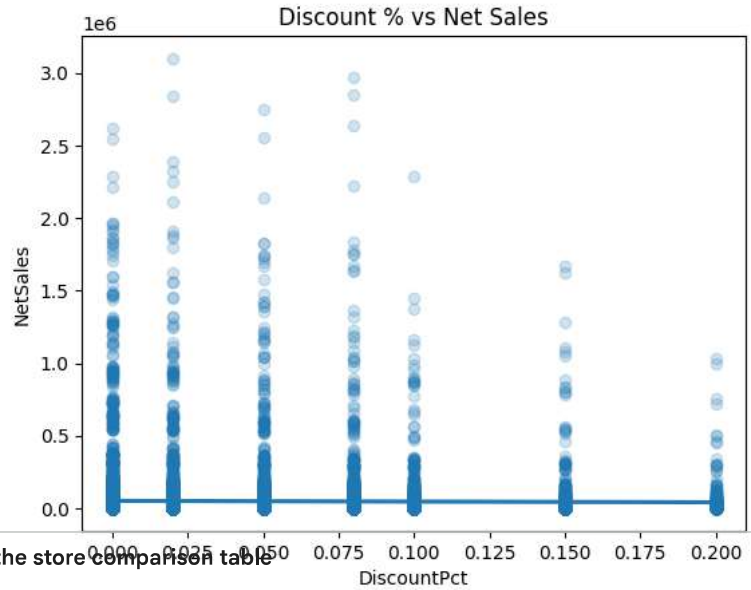
# Visual 1: discount vs net sales (scatter + trend)
sns.regplot(data=q2, x="DiscountPct", y="NetSales", scatter_kws={"alpha":0.2})
plt.title("Discount % vs Net Sales"); plt.show()

# Visual 2: bucket transactions by discount level
q2["DiscBand"] = pd.cut(q2.DiscountPct,
                        bins=[-0.001,0,0.05,0.10,0.15,0.20,1],
                        labels=["0%","0-5%","5-10%","10-15%","15-20%",">20%"])
band = q2.groupby("DiscBand").agg(
    AvgNetSales=("NetSales","mean"),
    AvgQty=("Quantity","mean"),
    AvgGrossProfit=("GrossProfit","mean"),
    N=("NetSales","size"))
print(band)
band.plot(kind="bar", subplots=True, figsize=(10,8)); plt.tight_layout(); plt.show()
```

```
        DiscountPct  DiscountAmount    GrossSales      NetSales      Quantity  \
count  24975.000000    25035.000000  2.503500e+04  2.503500e+04  25035.000000
mean       0.041958     2170.118272  5.147581e+04  4.930569e+04      2.592610
std        0.046212     9513.237568  1.523395e+05  1.461956e+05      1.897384
min        0.000000        0.000000  8.010000e+02  6.496000e+02      1.000000
25%        0.000000        0.000000  7.673500e+03  7.331740e+03      1.000000
50%        0.020000      342.480000  1.768100e+04  1.701000e+04      2.000000
75%        0.080000     1390.500000  4.048500e+04  3.858900e+04      3.000000
max        0.200000   295181.100000  3.230810e+06  3.099113e+06     10.000000

         GrossProfit
count   25035.000000
mean    -1627.396722
std     13888.364029
min   -360538.200000
25%     -1636.040000
50%      -386.540000
75%       103.150000
max    362670.000000
                DiscountPct  DiscountAmount  GrossSales  NetSales  Quantity  \
DiscountPct        1.000000        0.251188    0.000922 -0.015371  0.000553
DiscountAmount     0.251188        1.000000    0.664020  0.626853  0.166737
GrossSales         0.000922        0.664020    1.000000  0.998816  0.246910
NetSales          -0.015371        0.626853    0.998816  1.000000  0.246437
Quantity           0.000553        0.166737    0.246910  0.246437  1.000000
GrossProfit       -0.172160       -0.603369   -0.333275 -0.308019 -0.097622

                GrossProfit
DiscountPct       -0.172160
DiscountAmount    -0.603369
GrossSales        -0.333275
NetSales          -0.308019
Quantity          -0.097622
GrossProfit        1.000000
```



Q3: the store comparison table

```
st = sales_with_cost.merge(stores, on="StoreID", how="left")

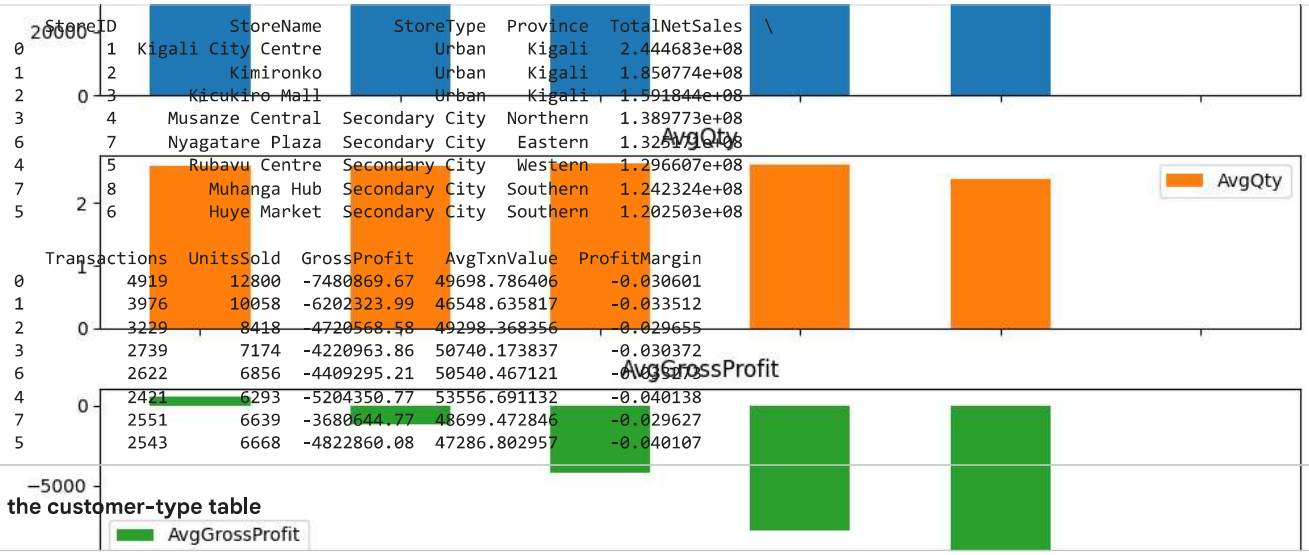
store = st.groupby(["StoreID","StoreName","StoreType","Province"]).agg(
```

```
    TotalNetSales = ("NetSales","sum"),
    Transactions   = ("TransactionID","nunique"),
    UnitsSold     = ("Quantity","sum"),
    GrossProfit   = ("GrossProfit","sum"),
).reset_index()

store["AvgTxnValue"]  = store.TotalNetSales / store.Transactions
store["ProfitMargin"] = store.GrossProfit / store.TotalNetSales
print(store.sort_values("TotalNetSales", ascending=False))
```

```
  StoreID          StoreName      StoreType  Province  TotalNetSales  \
0       1  Kigali City Centre          Urban    Kigali   2.444683e+08
1       2           Kimironko          Urban    Kigali   1.850774e+08
2       3       Kicukiro Mall          Urban    Kigali   1.591844e+08
3       4     Musanze Central  Secondary City  Northern   1.389773e+08
6       7     Nyagatare Plaza  Secondary City   Eastern   1.325173e+08
4       5       Rubavu Centre  Secondary City   Western   1.296607e+08
7       8        Muhanga Hub  Secondary City  Southern   1.242324e+08
5       6        Huye Market  Secondary City  Southern   1.202503e+08

   Transactions  UnitsSold  GrossProfit   AvgTxnValue  ProfitMargin
0          4919      12800  -7480869.67  49698.786406     -0.030601
1          3976      10058  -6202323.99  46548.635817     -0.033512
2          3229       8418  -4720568.58  49298.368356     -0.029655
3          2739       7174  -4220963.86  50740.173837     -0.030372
6          2622       6856  -4409295.21  50540.467121     -0.033273
4          2421       6293  -5204350.77  53556.691132     -0.040138
7          2551       6639  -3680644.77  48699.472846     -0.029627
5          2543       6668  -4822860.08  47286.802957     -0.040107
```



Q4: the customer-type table

```
customers = pd.read_excel(FILE, sheet_name="DimCustomer")

sales_customer_data = sales_with_cost.merge(customers[['CustomerID', 'CustomerType']], on='CustomerID', how='left')

cust = sales_customer_data.groupby("CustomerType").agg(
    Customers     = ("CustomerID","nunique"),
    Transactions  = ("TransactionID","nunique"),
    TotalNetSales = ("NetSales","sum"),
    TotalQty      = ("Quantity","sum"),
    GrossProfit   = ("GrossProfit","sum"),
).reset_index()

cust["PctNetSales"]   = cust.TotalNetSales / cust.TotalNetSales.sum()*100
cust["AvgTxnValue"]   = cust.TotalNetSales / cust.Transactions
cust["ProfitMargin"]  = cust.GrossProfit / cust.TotalNetSales
print(cust)
```

```
  CustomerType  Customers  Transactions  TotalNetSales  TotalQty  GrossProfit  \
0   Individual       3600         18146   8.926403e+08     47176 -30331535.14
1  Institution        256          1271   7.229076e+07      3350  -2352637.67
2          SME       1119          5583   2.694370e+08     14380  -8057704.12

   PctNetSales   AvgTxnValue  ProfitMargin
0    72.315569  49192.123050     -0.033980
1     5.856500  56877.075004     -0.032544
2    21.827931  48260.253606     -0.029906
```

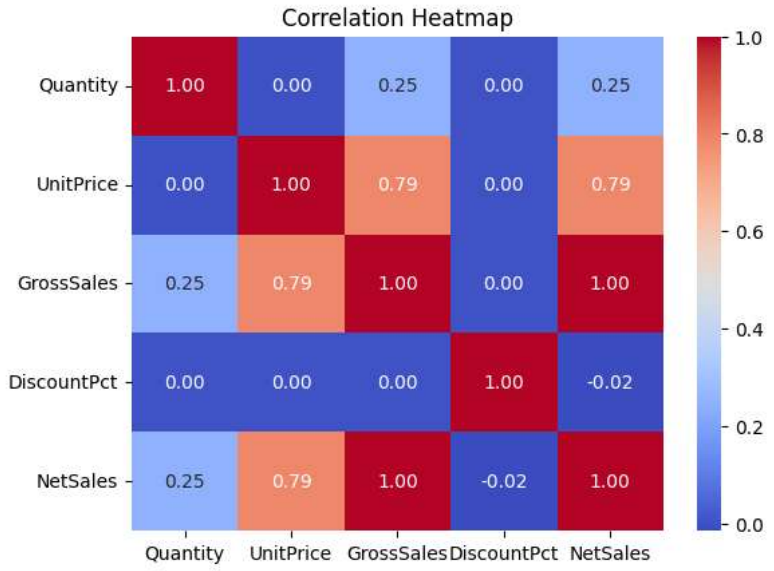
Q5: the correlation matrix

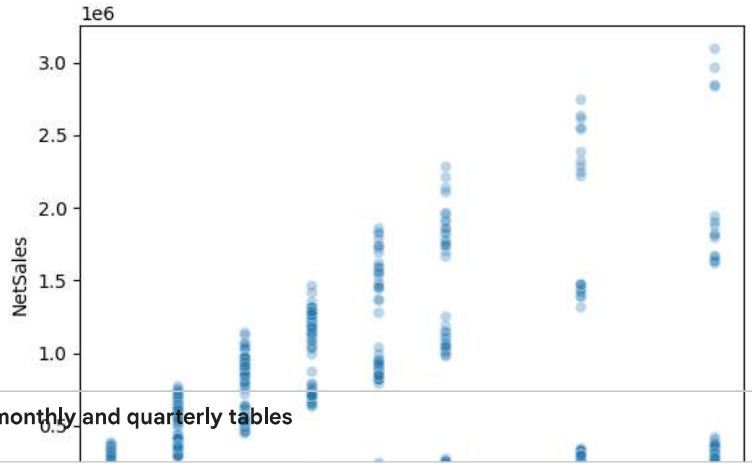
```
q5 = sales[["Quantity","UnitPrice","GrossSales","DiscountPct","NetSales"]].copy()
print(q5.describe())
corr = q5.corr(numeric_only=True)
print(corr)

sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Heatmap"); plt.show()

# Visual 1: Quantity vs NetSales
sns.scatterplot(data=q5, x="Quantity", y="NetSales", alpha=0.3); plt.show()
# Visual 2: UnitPrice vs NetSales (colored by discount)
sns.scatterplot(data=q5, x="UnitPrice", y="NetSales", hue="DiscountPct", alpha=0.4); plt.show()
```

```
           Quantity      UnitPrice    GrossSales   DiscountPct      NetSales
count  25035.000000   24985.000000  2.503500e+04  24975.000000  2.503500e+04
mean       2.592610   19806.217210  5.147581e+04      0.041958  4.930569e+04
std        1.897384   45961.762042  1.523395e+05      0.046212  1.461956e+05
min        1.000000     791.000000  8.010000e+02      0.000000  6.496000e+02
25%        1.000000    4204.000000  7.673500e+03      0.000000  7.331740e+03
50%        2.000000    8330.000000  1.768100e+04      0.020000  1.701000e+04
75%        3.000000   18089.000000  4.048500e+04      0.080000  3.858900e+04
max       10.000000  387691.000000  3.230810e+06      0.200000  3.099113e+06
             Quantity  UnitPrice  GrossSales  DiscountPct  NetSales
Quantity     1.000000   0.000259    0.246910     0.000553  0.246437
UnitPrice    0.000259   1.000000    0.789778     0.002740  0.788208
GrossSales   0.246910   0.789778    1.000000     0.000922  0.998816
DiscountPct  0.000553   0.002740    0.000922     1.000000 -0.015371
NetSales     0.246437   0.788208    0.998816    -0.015371  1.000000
```





Q6: monthly and quarterly tables

```
# ============================================================
# QUESTION 6 — Does the Company Have a Monthly Sales Pattern?
# ============================================================

import pandas as pd
import matplotlib.pyplot as plt

# ---------- 1. Load data ----------
sales   = pd.read_excel(FILE, sheet_name="SalesRaw")
dimdate = pd.read_excel(FILE, sheet_name="DimDate")

# Ensure datetime types
sales["TransactionDate"] = pd.to_datetime(sales["TransactionDate"])
dimdate["Date"]          = pd.to_datetime(dimdate["Date"])   # adjust name if needed

# ---------- 2. Merge sales with date dimension ----------
q6 = sales.merge(dimdate, left_on="TransactionDate", right_on="Date", how="left")

# ---------- 3. Monthly aggregates ----------
monthly = q6.groupby(["Year","MonthNumber","Month","YearMonth"]).agg(
    MonthlyNetSales=("NetSales","sum"),
```

```
    TransactionCount=("TransactionID","nunique"),
    QuantitySold=("Quantity","sum"),
).reset_index().sort_values(["Year","MonthNumber"])

monthly["AvgTransactionValue"] = monthly.MonthlyNetSales / monthly.TransactionCount
monthly["MoM_PctChange"]       = monthly.MonthlyNetSales.pct_change() * 100

print("===== MONTHLY AGGREGATES =====")
print(monthly)

# ---------- 4. Quarterly aggregates ----------
quarterly = q6.groupby(["Year","Quarter"]).agg(
    QuarterlyNetSales=("NetSales","sum"),
    Q_Transactions=("TransactionID","nunique"),
    Q_Quantity=("Quantity","sum"),
).reset_index().sort_values(["Year","Quarter"])

quarterly["QoQ_PctChange"] = quarterly.QuarterlyNetSales.pct_change() * 100

print("\n===== QUARTERLY AGGREGATES =====")
print(quarterly)

# ---------- 5. Headline answers ----------
print("\n===== HEADLINE ANSWERS =====")
print("Highest month:", monthly.loc[monthly.MonthlyNetSales.idxmax(),
      ["YearMonth","MonthlyNetSales"]].to_dict())
print("Lowest month :", monthly.loc[monthly.MonthlyNetSales.idxmin(),
      ["YearMonth","MonthlyNetSales"]].to_dict())
print("Best quarter :", quarterly.loc[quarterly.QuarterlyNetSales.idxmax(),
      ["Year","Quarter","QuarterlyNetSales"]].to_dict())
print("Largest + MoM:", monthly.loc[monthly.MoM_PctChange.idxmax(),
      ["YearMonth","MoM_PctChange"]].to_dict())
print("Largest - MoM:", monthly.loc[monthly.MoM_PctChange.idxmin(),
      ["YearMonth","MoM_PctChange"]].to_dict())

# ---------- 6. Visualization 1: Monthly Net Sales & Transaction Volume ----------
fig, ax1 = plt.subplots(figsize=(14,6))
ax1.plot(monthly.YearMonth, monthly.MonthlyNetSales, marker="o", color="steelblue")
ax1.set_xlabel("Month"); ax1.set_ylabel("Net Sales", color="steelblue")
ax1.tick_params(axis="x", rotation=45)

ax2 = ax1.twinx()
ax2.bar(monthly.YearMonth, monthly.TransactionCount, alpha=0.25, color="gray")
ax2.set_ylabel("Transaction Count", color="gray")

plt.title("Monthly Net Sales & Transaction Volume — 2025")
fig.tight_layout(); plt.show()

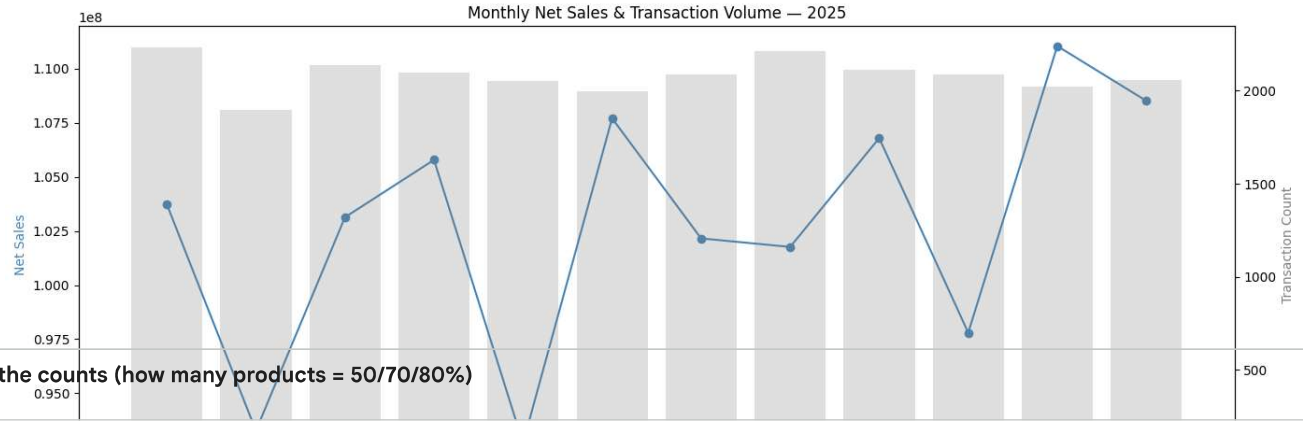
# ---------- 7. Visualization 2: Month-over-Month % Change ----------
plt.figure(figsize=(14,5))
plt.bar(monthly.YearMonth, monthly.MoM_PctChange, color="indianred")
plt.axhline(0, color="black", linewidth=0.8)
plt.xticks(rotation=45); plt.ylabel("MoM % Change")
plt.title("Month-over-Month % Change in Net Sales"); plt.show()
```

```
===== MONTHLY AGGREGATES =====
    Year  MonthNumber      Month YearMonth  MonthlyNetSales  TransactionCount  \
0   2025            1    January   2025-01     1.037253e+08              2235
1   2025            2   February   2025-02     9.327233e+07              1896
2   2025            3      March   2025-03     1.031316e+08              2138
3   2025            4      April   2025-04     1.057824e+08              2097
4   2025            5        May   2025-05     9.269844e+07              2052
5   2025            6       June   2025-06     1.077024e+08              1997
6   2025            7       July   2025-07     1.021554e+08              2086
7   2025            8     August   2025-08     1.017619e+08              2214
8   2025            9  September   2025-09     1.067885e+08              2114
9   2025           10    October   2025-10     9.780096e+07              2089
10  2025           11   November   2025-11     1.110400e+08              2023
11  2025           12   December   2025-12     1.085089e+08              2059

    QuantitySold  AvgTransactionValue  MoM_PctChange
0           5878         46409.551772            NaN
1           4861         49194.268465     -10.077590
2           5583         48237.405061      10.570379
3           5281         50444.628679       2.570323
4           5336         45174.679922     -12.368735
5           5262         53932.118438      16.185814
6           5379         48971.911779      -5.150331
7           5880         45962.899246      -0.385245
8           5504         50514.884456       4.939578
9           5415         46817.117391      -8.416178
10          5177         54888.756233      13.536673
11          5350         52699.783895      -2.279449

===== QUARTERLY AGGREGATES =====
   Year Quarter  QuarterlyNetSales  Q_Transactions  Q_Quantity  QoQ_PctChange
0  2025      Q1       3.001293e+08            6269       16322            NaN
1  2025      Q2       3.061833e+08            6146       15879       2.017137
2  2025      Q3       3.107057e+08            6414       16763       1.477044
3  2025      Q4       3.173498e+08            6171       15942       2.138369

===== HEADLINE ANSWERS =====
Highest month: {'YearMonth': '2025-11', 'MonthlyNetSales': 111039953.86}
Lowest month : {'YearMonth': '2025-05', 'MonthlyNetSales': 92698443.2}
Best quarter : {'Year': 2025, 'Quarter': 'Q4', 'QuarterlyNetSales': 317349767.13}
Largest + MoM: {'YearMonth': '2025-06', 'MoM_PctChange': 16.18581370091445}
Largest - MoM: {'YearMonth': '2025-05', 'MoM_PctChange': -12.368735091630755}
```



Q7: the counts (how many products = 50/70/80%)

```
sales_product_data = sales_with_cost.merge(products[['ProductID', 'ProductName', 'Category']], on='ProductID', how='left')

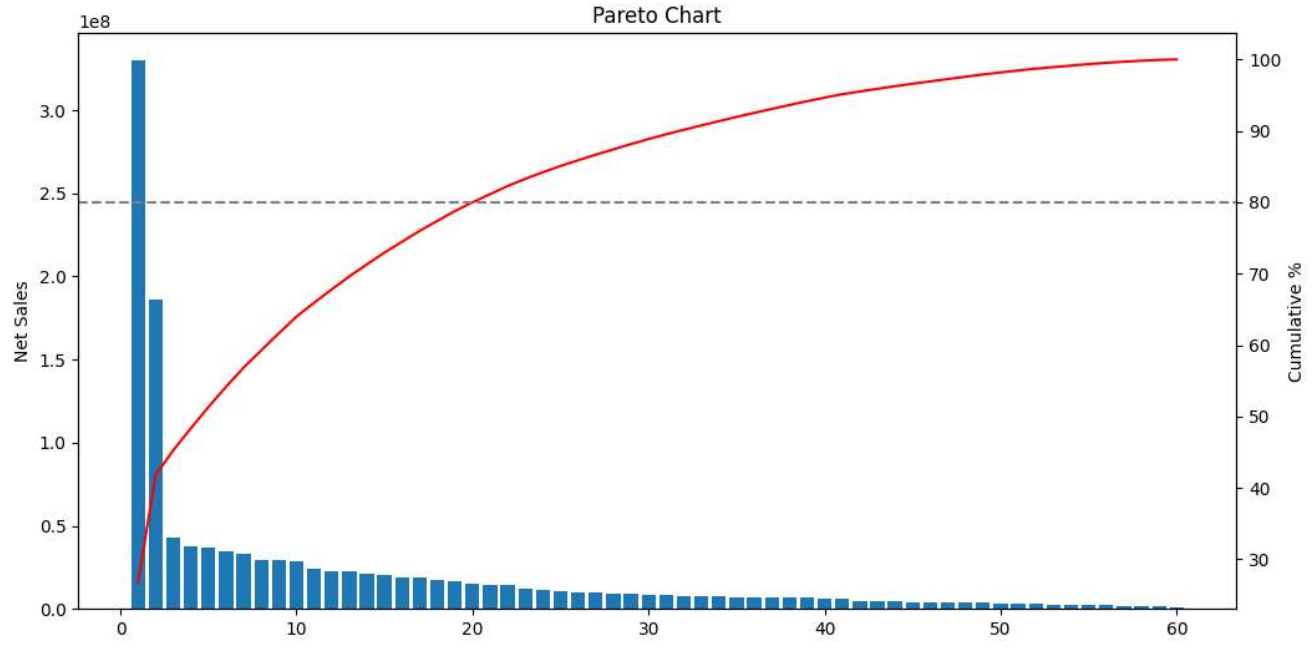
p = sales_product_data.groupby(["ProductID","ProductName","Category"])["NetSales"].sum() \
        .reset_index().sort_values("NetSales", ascending=False)
p["Pct"]       = p.NetSales / p.NetSales.sum()*100
p["CumPct"]    = p.Pct.cumsum()
p["Rank"]      = range(1, len(p)+1)

n50 = (p.CumPct <= 50).sum() + 1
n70 = (p.CumPct <= 70).sum() + 1
n80 = (p.CumPct <= 80).sum() + 1
print(f"Products for 50%: {n50}, 70%: {n70}, 80%: {n80} of {len(p)}")

fig, ax1 = plt.subplots(figsize=(12,6))
ax1.bar(p.Rank, p.NetSales); ax1.set_ylabel("Net Sales")
ax2 = ax1.twinx()
ax2.plot(p.Rank, p.CumPct, color="red"); ax2.axhline(80, ls="--", color="gray")
ax2.set_ylabel("Cumulative %"); plt.title("Pareto Chart"); plt.show()
```



```
Products for 50%: 5, 70%: 14, 80%: 21 of 60
```



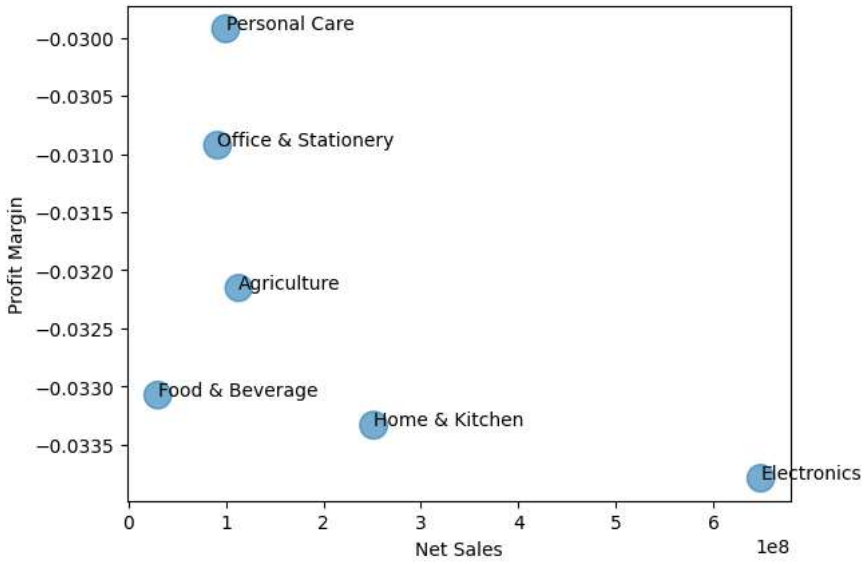
Q8: the category table

```
cat = sales_product_data.groupby("Category").agg(
    TotalNetSales = ("NetSales","sum"),
    Quantity       = ("Quantity","sum"),
    GrossProfit   = ("GrossProfit","sum"),
    NumProducts   = ("ProductID","nunique"),
).reset_index()
cat["PctNetSales"]  = cat.TotalNetSales / cat.TotalNetSales.sum()*100
cat["ProfitMargin"] = cat.GrossProfit / cat.TotalNetSales
print(cat)

# Bubble chart: sales vs margin, size = quantity
plt.scatter(cat.TotalNetSales, cat.ProfitMargin, s=cat.Quantity/50, alpha=0.6)
for _,r in cat.iterrows(): plt.annotate(r.Category,(r.TotalNetSales,r.ProfitMargin))
plt.xlabel("Net Sales"); plt.ylabel("Profit Margin"); plt.show()
```

```
              Category  TotalNetSales  Quantity  GrossProfit  NumProducts  \
0          Agriculture   1.129778e+08     10401  -3632683.50           10
1          Electronics   6.488274e+08     10762 -21924631.50           10
2      Food & Beverage   2.997370e+07     10763   -991402.67           10
3       Home & Kitchen   2.515526e+08     11075  -8385396.97           10
4  Office & Stationery   9.129694e+07     10858  -2823360.20           10
5        Personal Care   9.973960e+07     11047  -2984402.09           10

   PctNetSales  ProfitMargin
0     9.152685     -0.032154
1    52.563527     -0.033791
2     2.428263     -0.033076
3    20.379060     -0.033335
4     7.396250     -0.030925
5     8.080216     -0.029922
```



Q9: the store target-achievement table

```
sales["Month"] = pd.to_datetime(sales.TransactionDate).dt.to_period("M").astype(str)

# Convert 'Month' in targets to the same format as in sales before merging
targets['Month'] = pd.to_datetime(targets['Month']).dt.to_period("M").astype(str)

actual = sales.groupby(["Month","StoreID"])["NetSales"].sum().reset_index()
actual = actual.merge(targets, on=["Month","StoreID"], how="left")
actual = actual.merge(stores[["StoreID","StoreName"]], on="StoreID", how="left")

actual["Variance"]    = actual.NetSales - actual.SalesTarget
actual["Achievement"] = actual.NetSales / actual.SalesTarget * 100
print(actual)

# Who exceeded / missed
print(actual[actual.Variance > 0])
print(actual[actual.Variance < 0])

# By store
by_store = actual.groupby("StoreName").agg(
    TotalActual=("NetSales","sum"),
    TotalTarget=("SalesTarget","sum"),
    TotalVariance=("Variance","sum")).reset_index()
by_store["OverallAchievement"] = by_store.TotalActual / by_store.TotalTarget * 100
print(by_store.sort_values("OverallAchievement", ascending=False))
# Largest positive / negative variance
print(by_store.sort_values("TotalVariance", ascending=False).head(1))
print(by_store.sort_values("TotalVariance").head(1))

# Visualization
pivot = actual.pivot_table(index="Month", columns="StoreName",
                           values=["NetSales","SalesTarget"])
pivot.plot(kind="bar", figsize=(14,7)); plt.title("Actual vs Target by Store-Month")
plt.tight_layout(); plt.show()
```

```
      Month  StoreID     NetSales  SalesTarget          StoreName  \
0   2025-01        1  23563586.51     17464775  Kigali City Centre
1   2025-01        2  13072065.77     18074483          Kimironko
2   2025-01        3  14430248.64     23763965       Kicukiro Mall
3   2025-01        4  11546387.91     22645508     Musanze Central
4   2025-01        5   8305590.91     19988105       Rubavu Centre
..      ...      ...          ...          ...                 ...
91  2025-12        4  14453129.24     25634715     Musanze Central
92  2025-12        5  10984084.11     22626535       Rubavu Centre
93  2025-12        6   9582844.30     14897761        Huye Market
94  2025-12        7  10251493.41     23657590     Nyagatare Plaza
95  2025-12        8  13247729.14     26039268         Muhanga Hub

       Variance  Achievement
0    6098811.51   134.920642
1   -5002417.23    72.323318
2   -9333716.36    60.723236
3  -11099120.09    50.987542
4  -11682514.09    41.552668
..          ...          ...
91 -11181585.76    56.381080
92 -11642450.89    48.545144
93  -5314916.70    64.324057
94 -13406096.59    43.332788
95 -12791538.86    50.875966

[96 rows x 7 columns]
      Month  StoreID     NetSales  SalesTarget          StoreName  \
0   2025-01        1  23563586.51     17464775  Kigali City Centre
16  2025-03        1  18906529.42     17883929  Kigali City Centre
17  2025-03        2  18715576.53     18508271          Kimironko
40  2025-06        1  22893771.81     18512661  Kigali City Centre
48  2025-07        1  22945313.99     18722239  Kigali City Centre
56  2025-08        1  21639216.65     18931816  Kigali City Centre
64  2025-09        1  24156153.77     19141393  Kigali City Centre
80  2025-11        1  23476911.75     19560548  Kigali City Centre
88  2025-12        1  25191541.40     19770125  Kigali City Centre

      Variance  Achievement
0   6098811.51   134.920642
16  1022600.42   105.717985
17   207305.53   101.120070
40  4381110.81   123.665484
48  4223074.99   122.556463
56  2707400.65   114.300797
64  5014760.77   126.198515
80  3916363.75   120.021749
88  5421416.40   127.422267
      Month  StoreID     NetSales  SalesTarget        StoreName     Variance  \
1   2025-01        2  13072065.77     18074483        Kimironko  -5002417.23
2   2025-01        3  14430248.64     23763965    Kicukiro Mall  -9333716.36
3   2025-01        4  11546387.91     22645508  Musanze Central -11099120.09
4   2025-01        5   8305590.91     19988105    Rubavu Centre -11682514.09
5   2025-01        6   9793196.25     13160566      Huye Market  -3367369.75
..      ...      ...          ...          ...              ...          ...
91  2025-12        4  14453129.24     25634715  Musanze Central -11181585.76
92  2025-12        5  10984084.11     22626535    Rubavu Centre -11642450.89
93  2025-12        6   9582844.30     14897761      Huye Market  -5314916.70
94  2025-12        7  10251493.41     23657590  Nyagatare Plaza -13406096.59
95  2025-12        8  13247729.14     26039268      Muhanga Hub -12791538.86
```

Q10: the employee table and role averages

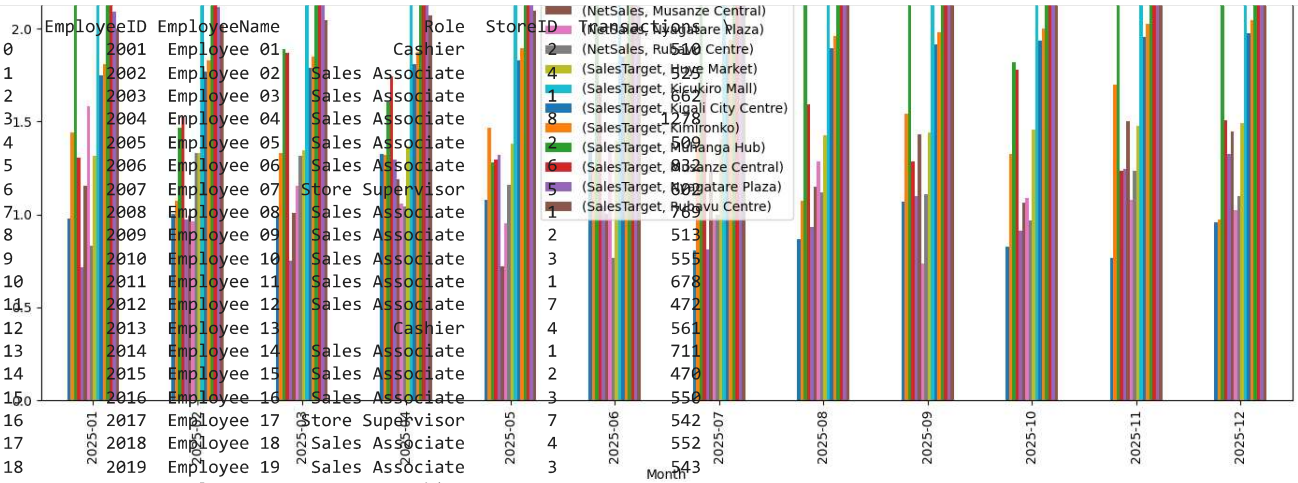
```
employees = pd.read_excel(FILE, sheet_name="DimEmployee")
sales_employee_data = sales_with_cost.merge(employees[['EmployeeID', 'EmployeeName', 'Role']], on='EmployeeID', how='left')

emp = sales_employee_data.groupby(["EmployeeID","EmployeeName","Role","StoreID"]).agg(
    Transactions  = ("TransactionID","nunique"),
    TotalNetSales = ("NetSales","sum"),
    TotalQty      = ("Quantity","sum"),
    GrossProfit   = ("GrossProfit","sum"),
).reset_index()

emp["AvgTxnValue"]  = emp.TotalNetSales / emp.Transactions
emp["ProfitMargin"] = emp.GrossProfit / emp.TotalNetSales
print(emp)

# Compare within roles
print(emp.groupby("Role")[["Transactions","TotalNetSales","AvgTxnValue","GrossProfit","ProfitMargin"]].mean())

# Top performers
print(emp.loc[emp.TotalNetSales.idxmax()])
print(emp.loc[emp.AvgTxnValue.idxmax()])
print(emp.loc[emp.GrossProfit.idxmax()])
```




```
    EmployeeID EmployeeName            Role  StoreID  Transactions  \
0         2001  Employee 01         Cashier        2           540
1         2002  Employee 02   Sales Associate        4           525
2         2003  Employee 03   Sales Associate        1           662
3         2004  Employee 04   Sales Associate        8          1278
4         2005  Employee 05   Sales Associate        2           509
5         2006  Employee 06   Sales Associate        6           832
6         2007  Employee 07  Store Supervisor        5           622
7         2008  Employee 08   Sales Associate        1           769
8         2009  Employee 09   Sales Associate        2           513
9         2010  Employee 10   Sales Associate        3           555
10        2011  Employee 11   Sales Associate        1           678
11        2012  Employee 12   Sales Associate        7           472
12        2013  Employee 13         Cashier        4           561
13        2014  Employee 14   Sales Associate        1           711
14        2015  Employee 15   Sales Associate        2           470
15        2016  Employee 16   Sales Associate        3           550
16        2017  Employee 17  Store Supervisor        7           542
17        2018  Employee 18   Sales Associate        4           552
18        2019  Employee 19   Sales Associate        3           543
19        2020  Employee 20         Cashier        1           688
20        2021  Employee 21  Store Supervisor        4           544
21        2022  Employee 22   Sales Associate        1           729
22        2023  Employee 23   Sales Associate        4           557
23        2024  Employee 24  Store Supervisor        3           516
24        2025  Employee 25   Sales Associate        6           848
25        2026  Employee 26  Store Supervisor        8          1273
26        2027  Employee 27  Store Supervisor        5           578
27        2028  Employee 28   Sales Associate        6           863
28        2029  Employee 29   Sales Associate        7           530
29        2030  Employee 30   Sales Associate        7           526
30        2031  Employee 31         Cashier        2           479
31        2032  Employee 32   Sales Associate        5           628
32        2033  Employee 33   Sales Associate        5           613
33        2034  Employee 34  Store Supervisor        2           518
34        2035  Employee 35   Sales Associate        2           469
35        2036  Employee 36   Sales Associate        2           508
36        2037  Employee 37         Cashier        3           540
37        2038  Employee 38   Sales Associate        3           525
38        2039  Employee 39         Cashier        1           682
39        2040  Employee 40         Cashier        7           552

    TotalNetSales  TotalQty  GrossProfit   AvgTxnValue  ProfitMargin
0     17090349.98      1243   -530150.02  33510.490157     -0.031020
1     29507510.42      1336  -1148689.58  56204.781752     -0.038929
2     35209959.12      1746  -1454840.88  53187.249426     -0.041319
3     64955495.93      3323  -1347304.07  50825.896659     -0.020742
4     22926957.30      1224  -1067142.70  45043.138114     -0.046545
5     37871278.80      2165  -1502321.20  45518.363942     -0.039669
6     29055090.09      1610  -1571609.91  48264.269252     -0.054091
7     40187279.00      2030  -1145621.00  52259.140442     -0.028507
8     25441749.80      1314   -898450.20  49594.054191     -0.035314
9     28162577.54      1404  -1068022.46  50743.382955     -0.037923
10    29351119.21      1693  -1228780.79  43290.736298     -0.041865
11    21043773.04      1282   -849226.96  44584.264915     -0.040355
12    22651281.43      1485   -614318.57  40376.615740     -0.027121
13    36461738.95      1890  -1429161.05  51282.333263     -0.039196
14    22054788.10      1240   -733111.90  46925.081064     -0.033240
```

```
import zipfile, glob
from google.colab import files

with zipfile.ZipFile("charts.zip", "w") as z:
    for img in glob.glob("*.png"):
        z.write(img)

files.download("charts.zip")
```

```
import zipfile, glob
from google.colab import files

with zipfile.ZipFile("assignment_outputs.zip", "w") as z:
    # tables
    if "all_outputs.xlsx" in glob.glob("*.xlsx"):
        z.write("all_outputs.xlsx")
    # text outputs
    for txt in glob.glob("*.txt"):
        z.write(txt)
    # charts
    for png in glob.glob("*.png"):
        z.write(png)
    # raw CSVs if you saved any
    for csv in glob.glob("*.csv"):
        z.write(csv)

files.download("assignment_outputs.zip")
```

```
# ============================================================
# KPI DASHBOARD — Numeric Money Columns
# NetSales, GrossProfit, SalesTarget
# ============================================================

import pandas as pd

# Define the file path
FILE = "/content/drive/MyDrive/Business_Intelligence/BI_Retail_Case_Study.xlsx"

# Load the required dataframes
sales   = pd.read_excel(FILE, sheet_name="SalesRaw")
products= pd.read_excel(FILE, sheet_name="DimProduct")
stores  = pd.read_excel(FILE, sheet_name="DimStore")
targets = pd.read_excel(FILE, sheet_name="Targets")

# --- Prep ---
sales["TransactionDate"] = pd.to_datetime(sales["TransactionDate"])
sales["Month"] = sales["TransactionDate"].dt.to_period("M").astype(str)

# Cost is required to derive GrossProfit if not already present
df = sales.merge(products[["ProductID", "UnitCost"]], on="ProductID", how="left")
df["GrossProfit"] = df["NetSales"] - (df["UnitCost"] * df["Quantity"])

# --- Aggregations ---
total_sales      = df["NetSales"].sum()
total_profit     = df["GrossProfit"].sum()
total_txn        = df["TransactionID"].nunique()
total_units      = df["Quantity"].sum()

monthly_sales    = df.groupby("Month")["NetSales"].sum()
monthly_mom      = monthly_sales.pct_change() * 100

top5_sales       = df.groupby("ProductID")["NetSales"].sum().nlargest(5).sum()
top5_profit      = df.groupby("ProductID")["GrossProfit"].sum().nlargest(5).sum()

store_sales      = df.merge(stores[["StoreID", "StoreName"]], on="StoreID", how="left") \
                     .groupby("StoreName")["NetSales"].sum()

# Targets — aggregate to same level as actual
target_total     = targets["SalesTarget"].sum()
actual_by_store  = df.groupby("StoreID")["NetSales"].sum()
target_by_store  = targets.groupby("StoreID")["SalesTarget"].sum()
ach_by_store     = (actual_by_store / target_by_store * 100)
```

```
# --- KPI dictionary ---
kpi = {
    # --- NetSales KPIs ---
    "Total Net Sales (RWF)":          total_sales,
    "Avg Transaction Value (ATV)":    total_sales / total_txn,
    "Monthly Avg Net Sales (RWF)":    monthly_sales.mean(),
    "Best Month":                     monthly_sales.idxmax(),
    "Best Month Net Sales (RWF)":     monthly_sales.max(),
    "Worst Month":                    monthly_sales.idxmin(),
    "Worst Month Net Sales (RWF)":    monthly_sales.min(),
    "Latest MoM Growth %":            monthly_mom.iloc[-1],
    "Top-5 Product Revenue Share %":  top5_sales / total_sales * 100,
    "Top Store Revenue Share %":      store_sales.max() / total_sales * 100,

    # --- GrossProfit KPIs ---
    "Total Gross Profit (RWF)":       total_profit,
    "Gross Profit Margin %":          total_profit / total_sales * 100,
    "Profit per Transaction (RWF)":   total_profit / total_txn,
    "Profit per Unit (RWF)":          total_profit / total_units,
    "Top-5 Product Profit Share %":   top5_profit / total_profit * 100 if total_profit != 0 else float("nan"),

    # --- SalesTarget KPIs ---
    "Total Sales Target (RWF)":       target_total,
    "Achievement % (Overall)":        total_sales / target_total * 100,
    "Variance vs Target (RWF)":       total_sales - target_total,
    "Variance vs Target %":           (total_sales - target_total) / target_total * 100,
    "% Stores Achieving Target":      (ach_by_store >= 100).mean() * 100,
}

kpi_df = pd.DataFrame(list(kpi.items()), columns=["KPI", "Value"])
print("=" * 60)
print("KPI DASHBOARD — MONEY COLUMNS")
print("=" * 60)

# Format nicely for readability
for _, row in kpi_df.iterrows():
    name = row["KPI"]
    val  = row["Value"]
    if isinstance(val, str):
        print(f"{name:<35} {val}")
    elif "Share" in name or "Margin" in name or "%" in name or "Growth" in name:
        print(f"{name:<35} {val:>15,.2f} %")
    else:
        print(f"{name:<35} {val:>15,.0f} RWF")
```

```
============================================================
KPI DASHBOARD — MONEY COLUMNS
============================================================
Total Net Sales (RWF)                 1,234,368,023 RWF
Avg Transaction Value (ATV)                  49,375 RWF
Monthly Avg Net Sales (RWF)             102,864,002 RWF
Best Month                              2025-11
Best Month Net Sales (RWF)              111,039,954 RWF
Worst Month                             2025-05
Worst Month Net Sales (RWF)              92,698,443 RWF
Latest MoM Growth %                           -2.28 %
Top-5 Product Revenue Share %                 51.32 %
Top Store Revenue Share %                     19.81 %
Total Gross Profit (RWF)                329,491,323 RWF
Gross Profit Margin %                         26.69 %
Profit per Transaction (RWF)                 13,180 RWF
Profit per Unit (RWF)                         5,076 RWF
Top-5 Product Profit Share %                  39.38 %
Total Sales Target (RWF)              2,033,918,030 RWF
Achievement % (Overall)                       60.69 %
Variance vs Target (RWF)              -799,550,007 RWF
Variance vs Target %                          -39.31 %
% Stores Achieving Target                     12.50 %
```

```
# ============================================================
# KPI DASHBOARD — Profit Margin
# ============================================================

import pandas as pd

# --- Prep ---
sales["TransactionDate"] = pd.to_datetime(sales["TransactionDate"])
```

```
sales["Month"] = sales["TransactionDate"].dt.to_period("M").astype(str)

# Merge in cost to compute GrossProfit
df = sales.merge(products[["ProductID", "UnitCost", "Category"]], on="ProductID", how="left")
df = df.merge(stores[["StoreID", "StoreName"]], on="StoreID", how="left")
cust = pd.read_excel(FILE, sheet_name="DimCustomer")
df = df.merge(cust[["CustomerID", "CustomerType"]], on="CustomerID", how="left")

df["GrossProfit"] = df["NetSales"] - (df["UnitCost"] * df["Quantity"])

# --- Helper: weighted margin at any level ---
def margin_pct(frame):
    ns = frame["NetSales"].sum()
    gp = frame["GrossProfit"].sum()
    return (gp / ns * 100) if ns != 0 else float("nan")

# --- Headline KPI ---
overall_margin = margin_pct(df)

# --- Segment margins ---
margin_by_store = df.groupby("StoreName").apply(margin_pct).sort_values()
margin_by_cat   = df.groupby("Category").apply(margin_pct).sort_values()
margin_by_cust  = df.groupby("CustomerType").apply(margin_pct).sort_values()

# --- Monthly margin + volatility ---
monthly_margin   = df.groupby("Month").apply(margin_pct)
monthly_std      = monthly_margin.std()

# --- KPI dictionary ---
kpi = {
    "Overall Gross Margin %":          overall_margin,
    "Best Store (Margin %)":           margin_by_store.idxmax(),
    "Best Store Margin Value":         margin_by_store.max(),
    "Worst Store (Margin %)":          margin_by_store.idxmin(),
    "Worst Store Margin Value":        margin_by_store.min(),
    "Best Category (Margin %)":        margin_by_cat.idxmax(),
    "Best Category Margin Value":      margin_by_cat.max(),
    "Best Customer Type (Margin %)":   margin_by_cust.idxmax(),
    "Best Customer Type Margin Value": margin_by_cust.max(),
    "Monthly Margin Volatility (σ %)": monthly_std,
    "Best Month (Margin %)":           monthly_margin.idxmax(),
    "Worst Month (Margin %)":          monthly_margin.idxmin(),
}

kpi_df = pd.DataFrame(list(kpi.items()), columns=["KPI", "Value"])
print("=" * 62)
print("KPI DASHBOARD — PROFIT MARGIN")
print("=" * 62)
for _, row in kpi_df.iterrows():
    name, val = row["KPI"], row["Value"]
    if isinstance(val, str):
        print(f"{name:<35} {val}")
    else:
        print(f"{name:<35} {val:>15,.2f} %")

# --- Detailed segment tables ---
print("\n--- Margin by Store ---")
print(margin_by_store.round(2).to_string())
print("\n--- Margin by Category ---")
print(margin_by_cat.round(2).to_string())
print("\n--- Margin by Customer Type ---")
print(margin_by_cust.round(2).to_string())
print("\n--- Margin by Month ---")
print(monthly_margin.round(2).to_string())
```

```
==============================================================
KPI DASHBOARD — PROFIT MARGIN
==============================================================
Overall Gross Margin %                        26.69 %
Best Store (Margin %)               Kigali City Centre
Best Store Margin Value                       27.16 %
Worst Store (Margin %)              Rubavu Centre
Worst Store Margin Value                      25.45 %
Best Category (Margin %)            Home & Kitchen
Best Category Margin Value                    33.81 %
Best Customer Type (Margin %)       SME
Best Customer Type Margin Value               27.07 %
Monthly Margin Volatility (σ %)                4.93 %
```

```
Best Month (Margin %)              2025-12
Worst Month (Margin %)             2025-02

--- Margin by Store ---
StoreName
Rubavu Centre         25.45
Huye Market           26.45
Nyagatare Plaza       26.55
Kimironko             26.77
Musanze Central       26.82
Kicukiro Mall         26.91
Muhanga Hub           26.93
Kigali City Centre    27.16

--- Margin by Category ---
Category
Electronics            21.40
Food & Beverage        25.81
Agriculture            31.44
Office & Stationery    32.63
Personal Care          32.66
Home & Kitchen         33.81

--- Margin by Customer Type ---
CustomerType
Institution    25.54
Individual     26.67
SME            27.07

--- Margin by Month ---
Month
2025-01    20.16
2025-02    19.61
2025-03    26.22
2025-04    25.06
2025-05    27.10
2025-06    25.56
2025-07    25.33
```